Data Analyst Agent

Ask questions about a CSV in plain English; get tables, charts and answers, with the code that produced them.

Project report

LLM: Groq · openai/gpt-oss-120b

Sample run: 2026-10-31 08:39 on sales.csv (1,500 rows)

1. Overview

The Data Analyst Agent lets someone with no coding skills explore a dataset. They upload a CSV and type a question such as "monthly revenue trend". An LLM plans the analysis, writes pandas or Plotly code, and runs it in a restricted sandbox. If the code fails, the agent reads the traceback and rewrites the code, up to three times. The answer appears as a table, a single number, text or an interactive chart, and the code is always one click away.

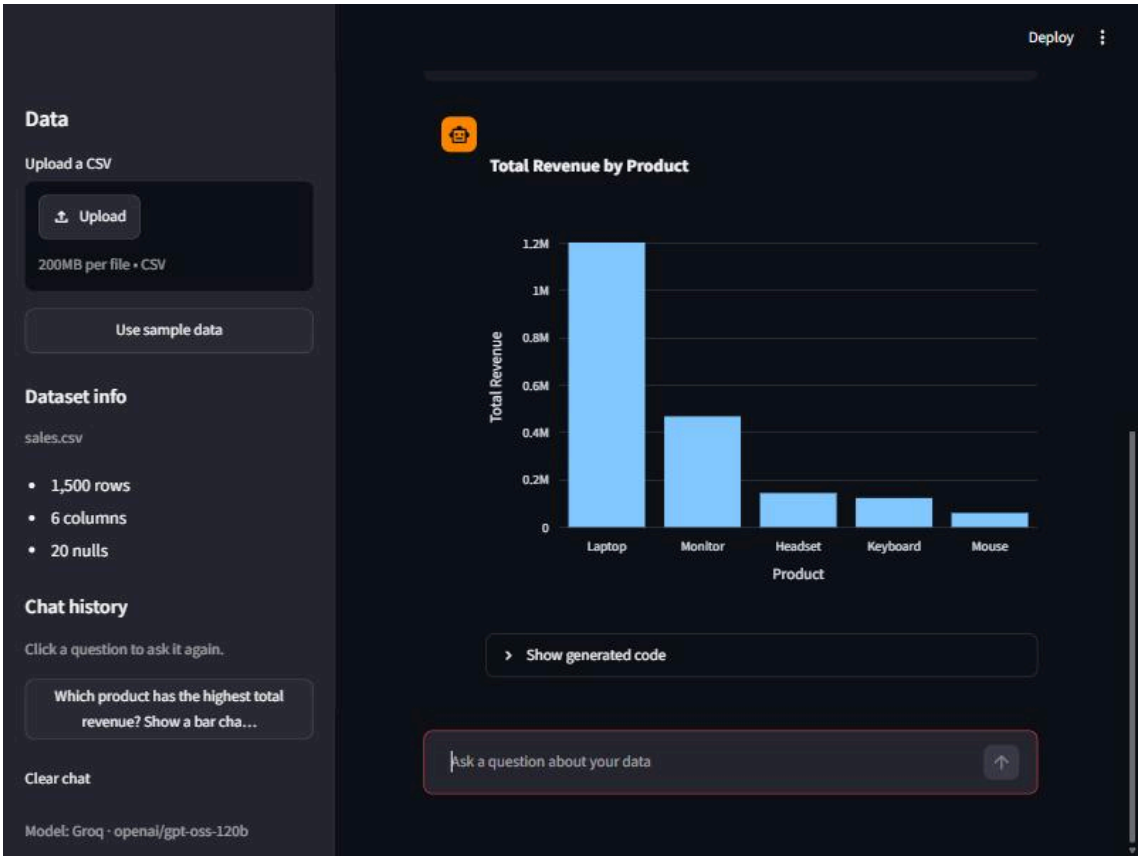
Key features

- **Natural language to code:** questions become pandas and Plotly code.
- **Self-healing:** failed code goes back to the LLM with its traceback, up to 3 fix attempts.
- **Automatic visualisation:** the code picks a chart when the question calls for one.
- **Schema-aware:** every call carries the column names, types, sample rows and statistics.
- **Code transparency:** the plan and the generated code are shown under every answer.
- **Follow-ups:** the last five turns, with their code, go into the prompts, so "now only 2024" works.

Results at a glance

- **9 of 9** sample questions answered, **9** of them with the first code.
- The self-correction demo recovered from a real KeyError in **1 fix attempt**.
- **112** automated tests cover the loader, prompts, sandbox, agent loop and app (no network needed).

THE STREAMLIT APP

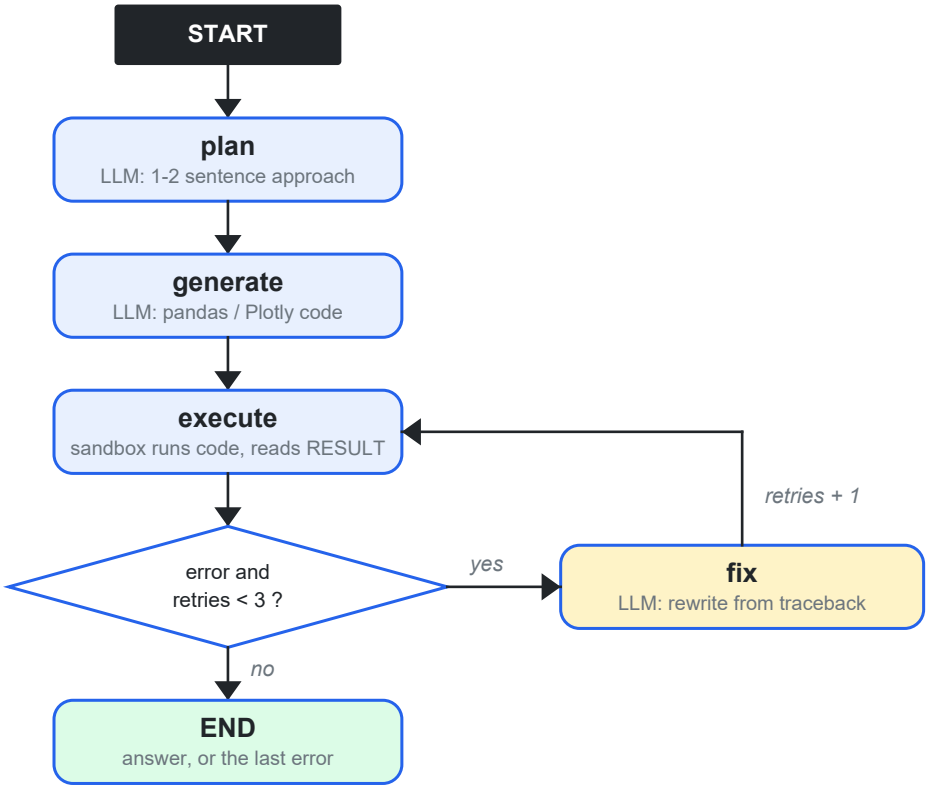


2. Architecture

Layer	Tool
Frontend	Streamlit
LLM	Groq (openai/gpt-oss-120b); Google Gemini as the alternative
Agent framework	LangGraph state machine
Data processing	pandas, NumPy
Charts	Plotly Express
Code execution	exec() with whitelisted builtins, an AST check and a timeout
Report	fpdf2, with charts exported by Kaleido

Agent graph

Every question runs through the same state machine. Each box is a node that reads the shared state and returns only the keys it changes.



Nodes

Node	What it does
plan	Asks the LLM for a one or two sentence approach, given the schema, the question and recent history.
generate	Asks for Python that uses df, pd, px and np and stores the answer in RESULT. Markdown fences and stray prose are stripped from the reply.

Node	What it does
execute	Runs the code in the sandbox on a copy of the data and classifies RESULT as a table, chart, value or text. A failure returns a trimmed traceback.
fix	Sends the traceback and the failed code back to the LLM for a corrected version; adds 1 to retries.
should_retry	Routes to fix while there is an error and fewer than 3 fixes have been tried; otherwise ends.

The sandbox

Generated code is checked and run with deliberately few powers:

- An **AST check** rejects imports, file access, eval/exec, dunder attributes and namespace tricks before anything runs, with a message the fix node can act on.
- **Whitelisted builtins** only (len, sum, sorted, round, and similar), so open() and the import function do not exist.
- The code gets a **copy** of the DataFrame, so a bad in-place edit cannot change the user's data.
- A **10-second timeout** stops runaway loops from blocking the app.

This guards against accidents, not attackers: it is not a security boundary.

LLM calls

One question costs at least two LLM calls (plan and generate), plus one per fix. Temporary failures (rate limits, 5xx errors, timeouts) are retried up to three times, waiting as long as the provider asks, and the app tells the user it is waiting. A wait longer than a minute, such as a used-up daily quota, stops at once with a message saying when the quota resets. Bad keys and unknown model names are reported without retrying.

3. Sample outputs

The spec's sample questions, asked of sales.csv (1,500 rows, 6 columns: date, product, region, revenue, quantity, salesperson). Each shows the agent's plan, its code and the result.

Q1. "Show the first 5 rows"

Table · 0 fix attempts · 2.2 s

Plan: I'll use pandas to read the CSV and call df.head(5) to display the first five rows as a table (no filtering, grouping, or aggregation needed). The answer will be presented as a simple tabular view of those rows.

```
RESULT = df.head(5)
```

date	product	region	revenue	quantity	salesperson
2023-01-01	Laptop	West	4,624.51	5	Ben
2023-01-02	Mouse	South	60.80	2	Elena
2023-01-02	Laptop	South	4,619.01	5	Dev
2023-01-03	Mouse	East	180.02	6	Farid
2023-01-03	Mouse	East	30.29	1	Carla

Q2. "What's the total revenue?"

Single value · 0 fix attempts · 1.5 s

Plan: I'll load the CSV into a pandas DataFrame and compute the total revenue with a simple aggregation:

total_revenue = df['revenue'].sum()

Since the question asks for a single overall total, I'll return the resulting number as a single-value answer.

```
RESULT = round(df['revenue'].sum(), 2)
```

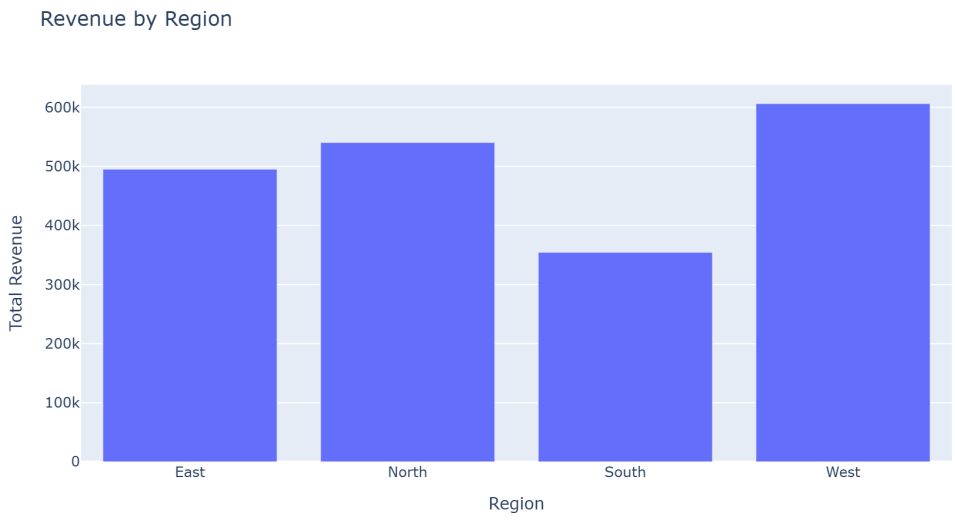
1,996,526.43

Q3. "Revenue by region as a bar chart"

Chart (bar) · 0 fix attempts · 1.8 s

Plan: I'll group the data by region, sum the revenue for each region (ignoring any missing values), and then feed the resulting totals into Plotly Express's px.bar to produce a bar chart of revenue by region. No filtering is needed beyond the aggregation step.

```
region_rev = df.groupby('region', as_index=False)['revenue'].sum()
RESULT = px.bar(
    region_rev,
    x='region',
    y='revenue',
    title='Revenue by Region',
    labels={'region': 'Region', 'revenue': 'Total Revenue'}
)
```



Q4. "Top 3 salespersons by quantity sold"

Table · 0 fix attempts · 1.7 s

Plan: I'll use the salesperson and quantity columns.
1. Drop rows where salesperson or quantity is null.
2. Group the data by salesperson and sum the quantity sold for each.
3. Sort the summed quantities in descending order and keep the first three rows.

The result will be presented as a table showing the three salespersons and their total quantities sold.

```
# Drop rows with missing salesperson or quantity
clean_df = df.dropna(subset=["salesperson", "quantity"])

# Aggregate total quantity per salesperson
agg = clean_df.groupby("salesperson")["quantity"].sum().reset_index()

# Get top 3 salespersons by quantity sold
RESULT = agg.sort_values(by="quantity", ascending=False).head(3).reset_index(drop=True)
```

salesperson	quantity
Ben	1,497
Asha	1,433
Farid	1,310

Q5. "Monthly revenue trend"

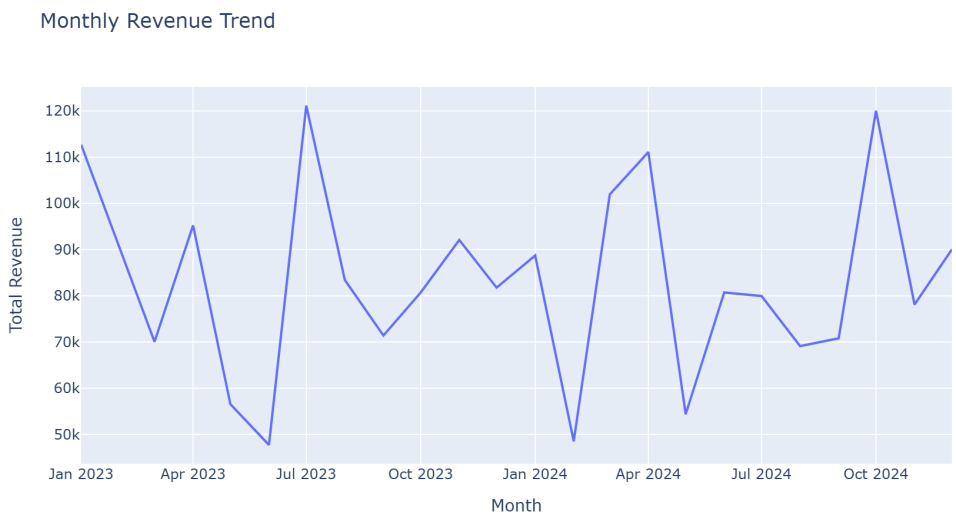
Chart (line) · 0 fix attempts · 2.2 s

Plan: I will use the date column (converted to year-month) and the revenue column.
1. Convert date to a period like date.dt.to_period('M') (or format as “YYYY-MM”).
2. Group the data by that month period and aggregate revenue with sum().
3. Feed the resulting month-wise totals to Plotly Express’s px.line() to produce a line chart showing the monthly revenue trend (x-axis = month, y-axis = total revenue).

The answer will be presented as a line-chart visualizing how revenue changes month-by-month.

```
monthly_rev = (
    df.groupby(df["date"].dt.to_period("M").dt.to_timestamp())["revenue"]
    .sum()
    .reset_index()
    .rename(columns={"date": "month", "revenue": "total_revenue"})
)

RESULT = px.line(
    monthly_rev,
    x="month",
    y="total_revenue",
    title="Monthly Revenue Trend",
    labels={"month": "Month", "total_revenue": "Total Revenue"},
)
```



Q6. "Now filter that to 2024 only"

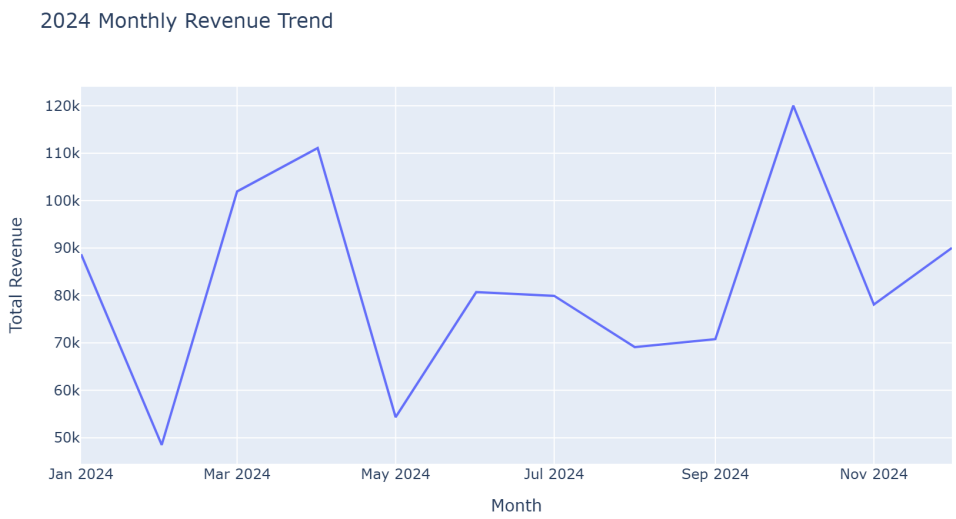
Chart (line) · 0 fix attempts · 2.6 s · follow-up to the monthly trend

Plan: I'll filter the dataframe to rows where df["date"].dt.year == 2024, then reuse the same grouping-by-month logic on the revenue column (summing it for each month) and plot the result with Plotly Express px.line. The answer will be an updated line chart titled "2024 Monthly Revenue Trend".

```
df_2024 = df[df["date"].dt.year == 2024]

monthly_rev = (
    df_2024.groupby(df_2024["date"].dt.to_period("M").dt.to_timestamp())["revenue"]
    .sum()
    .reset_index()
    .rename(columns={"date": "month", "revenue": "total_revenue"})
)

RESULT = px.line(
    monthly_rev,
    x="month",
    y="total_revenue",
    title="2024 Monthly Revenue Trend",
    labels={"month": "Month", "total_revenue": "Total Revenue"},
)
```



Q7. "Which product has the highest average revenue?"

Text answer · 0 fix attempts · 8.1 s

Plan: I'll load the CSV into a pandas DataFrame, then:

1. Select columns – use product and revenue.
2. Group & aggregate – df.groupby('product')['revenue'].mean() to compute the average revenue for each product.
3. Identify the maximum – pick the product with the highest mean value (e.g., using .idxmax()).

The answer will be a short sentence naming the product that has the highest average revenue (e.g., "Laptop has the highest average revenue"). No chart or table is needed.

```
avg_rev = df.groupby('product')['revenue'].mean()
max_product = avg_rev.idxmax()
max_value = avg_rev.max()
RESULT = f"{max_product} has the highest average revenue ({max_value:,.2f})."
```

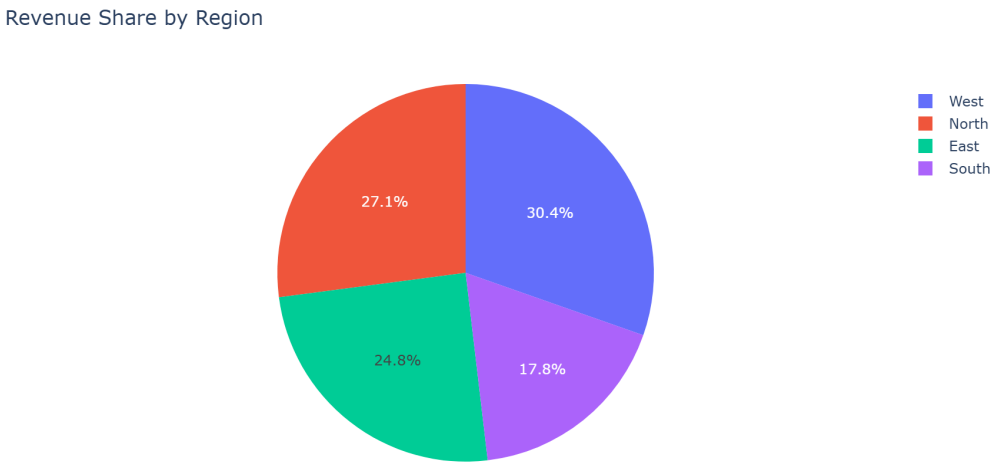
Laptop has the highest average revenue (5,031.41).

Q8. "Compare regions with a pie chart"

Chart (pie) · 0 fix attempts · 8.3 s

Plan: I'll aggregate the data by region, summing the revenue column (ignoring any missing values), and then feed the resulting region-revenue series to Plotly Express's px.pie to produce a pie chart that shows each region's share of total revenue. No filtering beyond dropping the few null rows is needed.

```
region_rev = df.groupby('region', as_index=False)['revenue'].sum()
RESULT = px.pie(
    region_rev,
    names='region',
    values='revenue',
    title='Revenue Share by Region',
    labels={'region': 'Region', 'revenue': 'Revenue'}
)
```



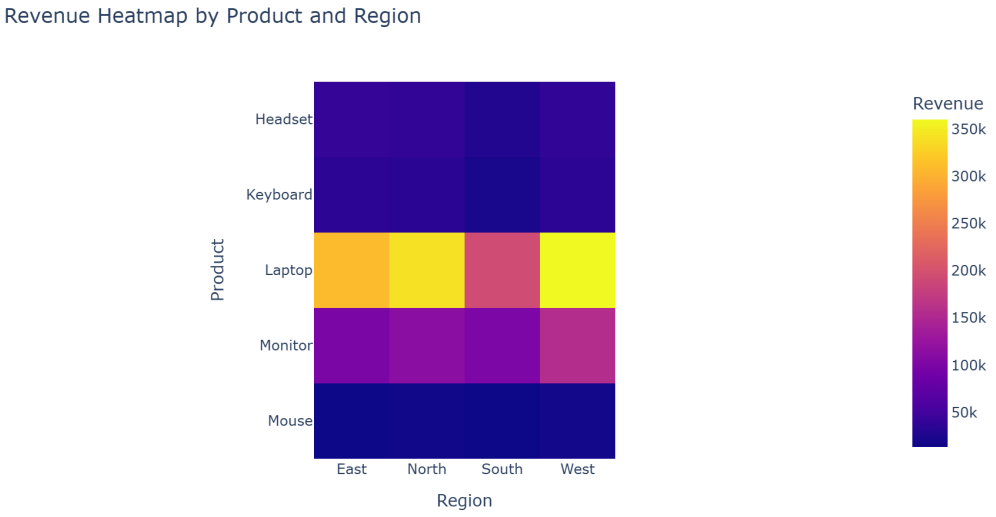
Q9. "Show a heatmap of revenue by product and region"

Chart (heatmap) · 0 fix attempts · 9.9 s

Plan: I'll aggregate the data with pandas by grouping on product and region, summing the revenue for each combination (ignoring the few missing values), and then feed that pivot-table into Plotly Express's px.imshow (or px.density_heatmap) to produce a heatmap chart of revenue by product × region. The answer will be a heatmap figure.

```
pivot = (
    df.groupby(["product", "region"], as_index=False)["revenue"]
    .sum()
    .pivot(index="product", columns="region", values="revenue")
)

RESULT = px.imshow(
    pivot,
    labels=dict(x="Region", y="Product", color="Revenue"),
    title="Revenue Heatmap by Product and Region",
)
```



4. Self-correction in action

On clean data the model rarely writes failing code, so the loop is hard to see. For this demo the first code was replaced with a version that uses a misspelled column (revenu). Everything after that is real: the sandbox raised a genuine KeyError, and the live model's fix node read the traceback and repaired the code.

QUESTION: TOTAL REVENUE BY PRODUCT, HIGHEST FIRST

1. FIRST CODE (BROKEN ON PURPOSE)

```
RESULT = df.groupby('product')['revenu'].sum().sort_values(ascending=False)
```

2. WHAT THE SANDBOX RETURNED TO THE FIX NODE

```
Traceback (most recent call last):
  File "<generated code>", line 1, in <module>
    RESULT = df.groupby('product')['revenu'].sum().sort_values(ascending=False)
             ~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^
  File "pandas/core/base.py", line 241, in __getitem__
    raise KeyError(f"Column not found: {key}")
KeyError: 'Column not found: revenu'
```

3. CODE AFTER 1 FIX ATTEMPT

```
RESULT = df.groupby('product')['revenue'].sum().sort_values(ascending=False).reset_index()
```

4. RESULT

product	revenue
Laptop	1,202,506.99
Monitor	467,745.69
Headset	143,747.43
Keyboard	122,904.31
Mouse	59,622.01

5. Limitations

- **The sandbox is not a security boundary.** It blocks accidental damage, but exec() inside the app's process cannot stop a determined attacker. Do not expose the app publicly with untrusted users.
- **A timed-out run keeps going in the background.** Python threads cannot be killed, so the 10-second limit frees the app but not the CPU.
- **Wrong but working answers pass.** The retry loop catches code that crashes, not code that answers a different question. Showing the plan and the code is the safeguard.
- **One CSV at a time,** and the LLM sees only a summary of it (schema, sample rows, statistics).
- **Free-tier limits.** Groq's free tier limits tokens per minute, so a fast series of questions waits a few seconds; Gemini's free key allowed only 20 requests a day per model.

6. Learnings

- **Restricting exec() takes more than a small globals dict.** Python adds the full builtins unless the code's builtins are replaced, so open() and the import function stay available. A whitelist plus an AST check closed that.
- **Models do not follow format rules reliably.** Replies arrive in markdown fences, with prose around the code, or with print() calls. Cleaning the reply before running it removed a whole class of failures.
- **Errors are prompts.** Sending a short traceback (the last 30 lines) back to the model is usually enough for it to fix a wrong column name or type in one attempt.
- **Follow-ups need the previous code,** not just the previous question. History holds each turn's code and a one-line summary of its result.
- **Free-tier models move.** gemini-2.5-flash closed to new keys and the free quota was 20 requests a day, which ran out during testing. Keeping the model name in settings, and adding Groq as a second provider behind the same retry and error handling, kept the project working.
- **Test the loop without the network.** A scripted fake LLM made the retry routing, history and error handling testable in seconds, with no API key.